

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology and Engineering — DEPSTAR-IT

Course Code & Name: ITUE301 — Advanced Web Development Frameworks

Academic Year: AY 2026–27 | **Semester:** 5th

Examination: Open-Book Practical Examination (SET B)

Project Title: Library Book Management System

Student Details

Student Name: Shafin Nigamana

Roll Number / ID: d25dit094

Batch: C

GitHub Repository URL: <https://github.com/ShafinNigamana/itue301-exam-d25dit094-5itC>

Commit ID (SHA): 1b8699906cab67185f269f6569cfbfa9dc73530b

1. Project Overview & Architecture

The **Library Book Management System** is a full-stack web application designed to digitize basic information about library books, registered members, and borrowing records.

Technology Stack

Frontend: React.js, React Router DOM, Axios / Fetch API, CSS3

Backend: Node.js, Express.js.

Database: MongoDB with Mongoose ODM

Testing & Tools: Postman / Thunder Client, MongoDB Compass / Atlas, Git & GitHub

2. Implementation Summary

Task 1 & 2: Frontend Architecture, Routing & Forms

Task 3: Express REST API & Global Middleware

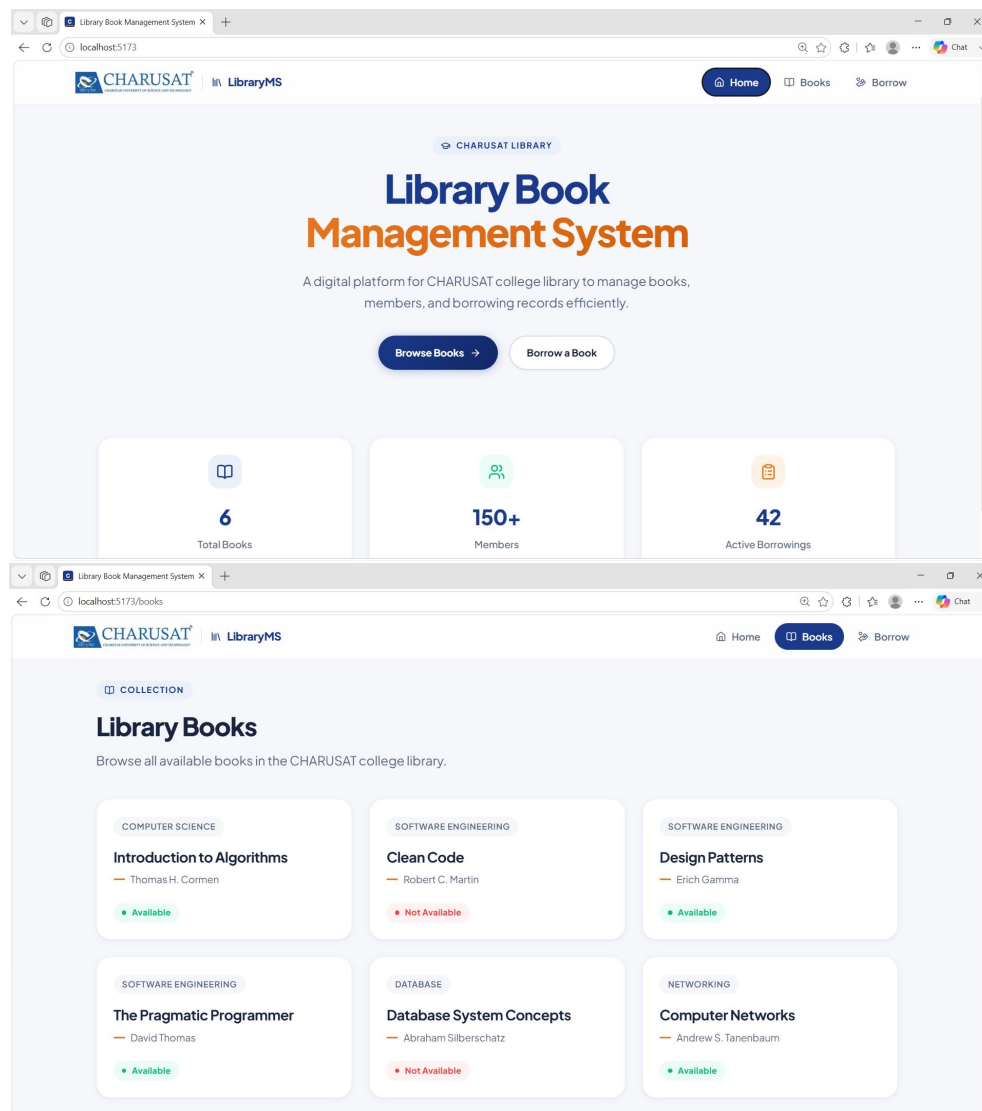
Task 4: REST API Consumption in React

Task 5: Database Schema Design & Validation

2. Practical Output

Screenshot 1 — React Application Frontend

Caption: React frontend running in the browser showing the navigation bar, book listing with availability status indicators, and the dynamic stateful borrowing form.



The screenshot shows a web browser window with the URL `localhost:5173/borrow`. The page is titled "Borrow a Book" and includes a sub-header "Fill in the details below to create a new borrowing record." The form is divided into two main sections: "Borrowing Details" and "Live Preview".

Borrowing Details:

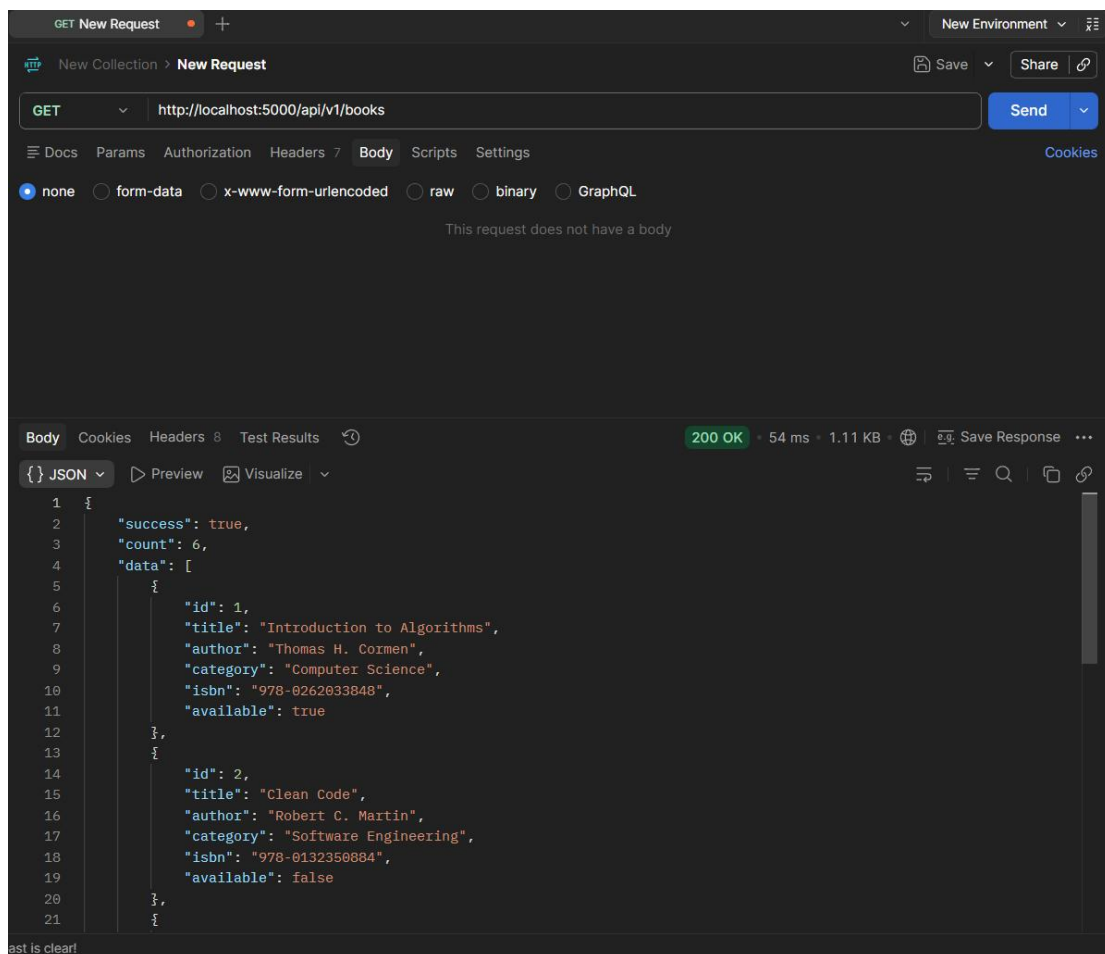
- Member Name:
- Book Title:
- Borrow Date:
- Return Date:

Live Preview:

- MEMBER NAME: Shafin Nigamana
- BOOK TITLE: Harry Potter
- BORROW DATE: 2026-08-19
- RETURN DATE: 2026-08-21

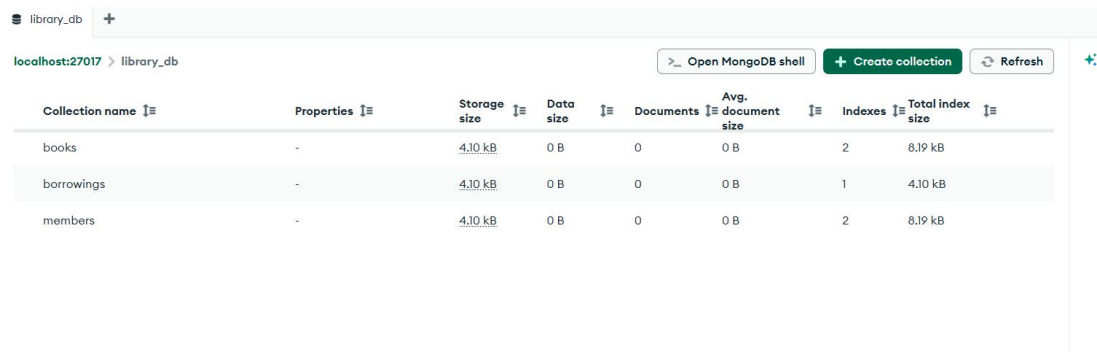
Screenshot 2 — Express REST API Execution

Caption: API endpoint execution in Postman / Thunder Client demonstrating a successful request and structured JSON response with proper HTTP status code.



Screenshot 3 — MongoDB Document Verification

Caption: MongoDB Compass / Atlas interface displaying the successfully created document records adhering to the Mongoose schema definitions.

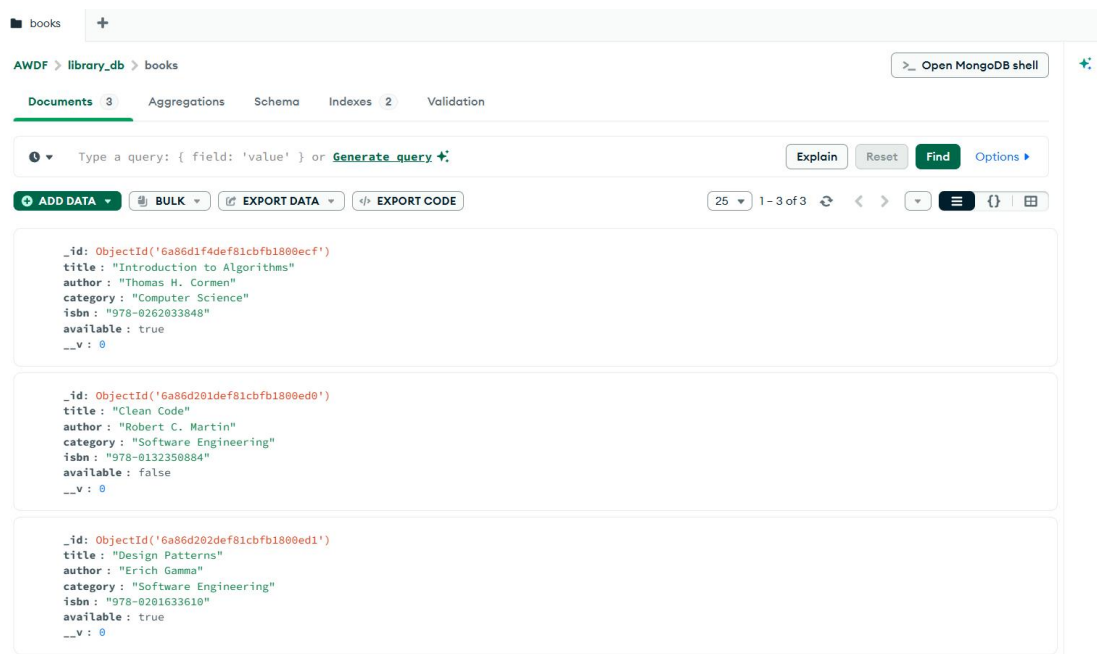


library_db

localhost:27017 > library_db

Open MongoDB shell Create collection Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
books	-	4.10 kB	0 B	0	0 B	2	8.19 kB
borrowings	-	4.10 kB	0 B	0	0 B	1	4.10 kB
members	-	4.10 kB	0 B	0	0 B	2	8.19 kB



books

AWDF > library_db > books

Open MongoDB shell

Documents 3 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or Generate query

ADD DATA BULK EXPORT DATA EXPORT CODE

25 1 - 3 of 3

<pre>_id: ObjectId('6a86d1f4def81cbfb1808ecf') title: "Introduction to Algorithms" author: "Thomas H. Cormen" category: "Computer Science" isbn: "978-0262033848" available: true __v: 0</pre>
<pre>_id: ObjectId('6a86d201def81cbfb1808ed0') title: "Clean Code" author: "Robert C. Martin" category: "Software Engineering" isbn: "978-0132350884" available: false __v: 0</pre>
<pre>_id: ObjectId('6a86d202def81cbfb1808ed1') title: "Design Patterns" author: "Erich Gamma" category: "Software Engineering" isbn: "978-0201633610" available: true __v: 0</pre>